

1)

As funções do Kernel podem ser divididas em cinco partes principais:

1. **Gerenciamento de Processos:** O kernel é responsável pela criação e destruição de processos, bem como pela conexão deles com o mundo exterior (entrada e saída). Ele lida com a comunicação entre diferentes processos (através de sinais, pipes ou primitivas de comunicação interprocessos) e inclui o agendador (*scheduler*), que controla como os processos compartilham a CPU.
2. **Gerenciamento de Memória:** O kernel constrói um espaço de endereçamento virtual para todos os processos sobre os recursos limitados disponíveis. Essa parte é crítica para o desempenho do sistema e interage através de chamadas de função para alocação e liberação de memória.
3. **Sistemas de Arquivos:** Como o Unix trata quase tudo como um arquivo, o kernel constrói sistemas de arquivos estruturados em cima de um hardware não estruturado. O Linux suporta múltiplos tipos de sistemas de arquivos para organizar dados no meio físico (como discos).
4. **Controle de Dispositivos:** Quase toda operação do sistema mapeia eventualmente para um dispositivo físico. O kernel deve conter *device drivers* (drivers de dispositivo) para cada periférico presente no sistema (discos rígidos, teclados, etc.) para controlar essas operações.
5. **Rede:** O sistema operacional deve gerenciar as operações de rede (pacotes de dados) porque elas são eventos assíncronos não específicos a um processo. O kernel coleta, identifica e despacha pacotes de entrada, além de lidar com o roteamento e a resolução de endereços.

2)

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/sched.h> // Define task_struct e init_task
#include <linux/init.h>
#include <linux/list.h> // Define list_head, list_for_each, list_entry

MODULE_LICENSE("GPL");

static int __init listar_processos_init(void)
{
    struct task_struct *task;
    struct list_head *list;

    printk(KERN_INFO "--- Inicio da Lista de Processos ---\n");

    list_for_each(list, &init_task.tasks)
    {
        task = list_entry(list, struct task_struct, tasks);

        printk(KERN_INFO "PID: %d | Processo: %s\n", task->pid, task-
>comm);
    }

    printk(KERN_INFO "--- Fim da Lista de Processos ---\n");

    return 0;
}
```

```

}

static void __exit listar_processos_exit(void)
{
    printk(KERN_INFO "Modulo de lista de processos removido.\n");
}

module_init(listar_processos_init);
module_exit(listar_processos_exit);

```

3)

```

#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/fs.h>
#include <linux/cdev.h>
#include <linux/uaccess.h>
#include <linux/slab.h>
#include <linux/list.h>
#include <linux/mutex.h>
#include <linux/device.h>
#include <linux/version.h> /* Adicionado para verificar versão se
necessário */

#define DEVICE_NAME "list_device"
#define CLASS_NAME "list_class"

MODULE_LICENSE("GPL");

struct data_node
{
    char *message;
    int size;
    struct list_head list;
};

static int majorNumber;
static struct class *listClass = NULL;
static struct device *listDevice = NULL;
static struct cdev listCdev;

static LIST_HEAD(msg_list);
static DEFINE_MUTEX(list_mutex);

static int dev_open(struct inode *, struct file *);
static int dev_release(struct inode *, struct file *);
static ssize_t dev_read(struct file *, char *, size_t, loff_t *);
static ssize_t dev_write(struct file *, const char *, size_t, loff_t *);

static struct file_operations fops =

```

```

{
    .open = dev_open,
    .read = dev_read,
    .write = dev_write,
    .release = dev_release,
};

static int __init list_driver_init(void)
{
    dev_t dev_no;
    int ret;

    ret = alloc_chrdev_region(&dev_no, 0, 1, DEVICE_NAME);
    if (ret < 0)
    {
        printk(KERN_ALERT "ListDriver: Falha ao alocar major number\n");
        return ret;
    }
    majorNumber = MAJOR(dev_no);

    #if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 4, 0)
        listClass = class_create(CLASS_NAME);
    #else
        listClass = class_create(THIS_MODULE, CLASS_NAME);
    #endif

    if (IS_ERR(listClass))
    {
        unregister_chrdev_region(dev_no, 1);
        return PTR_ERR(listClass);
    }

    cdev_init(&listCdev, &fops);
    ret = cdev_add(&listCdev, dev_no, 1);
    if (ret < 0)
    {
        class_destroy(listClass);
        unregister_chrdev_region(dev_no, 1);
        return ret;
    }

    listDevice = device_create(listClass, NULL, dev_no, NULL, DEVICE_NAME);
    if (IS_ERR(listDevice))
    {
        cdev_del(&listCdev);
        class_destroy(listClass);
        unregister_chrdev_region(dev_no, 1);
        return PTR_ERR(listDevice);
    }

    printk(KERN_INFO "ListDriver: Modulo carregado. Dispositivo criado em
/dev/%s\n", DEVICE_NAME);
    return 0;
}

```

```

static void __exit list_driver_exit(void)
{
    struct data_node *ptr, *next;

    mutex_lock(&list_mutex);
    list_for_each_entry_safe(ptr, next, &msg_list, list)
    {
        kfree(ptr->message);
        list_del(&ptr->list);
        kfree(ptr);
    }
    mutex_unlock(&list_mutex);

    device_destroy(listClass, MKDEV(majorNumber, 0));
    cdev_del(&listCdev);
    class_unregister(listClass);
    class_destroy(listClass);
    unregister_chrdev_region(MKDEV(majorNumber, 0), 1);

    printk(KERN_INFO "ListDriver: Modulo descarregado.\n");
}

static int dev_open(struct inode *inodep, struct file *filep)
{
    return 0;
}

static int dev_release(struct inode *inodep, struct file *filep)
{
    return 0;
}

static ssize_t dev_read(struct file *filep, char *buffer, size_t len,
loff_t *offset)
{
    struct data_node *entry;
    int bytes_read = 0;
    const char *msg_vazia = "vazia\n";
    int msg_len;

    if (mutex_lock_interruptible(&list_mutex))
        return -ERESTARTSYS;

    if (list_empty(&msg_list))
    {
        msg_len = strlen(msg_vazia);
        if (copy_to_user(buffer, msg_vazia, msg_len) != 0)
        {
            mutex_unlock(&list_mutex);
            return -EFAULT;
        }
        bytes_read = msg_len;
    }
}

```

```

else
{
    entry = list_first_entry(&msg_list, struct data_node, list);
    msg_len = entry->size;

    if (len < msg_len)
        msg_len = len;

    if (copy_to_user(buffer, entry->message, msg_len) != 0)
    {
        mutex_unlock(&list_mutex);
        return -EFAULT;
    }

    list_del(&entry->list);
    kfree(entry->message);
    kfree(entry);

    bytes_read = msg_len;
}

mutex_unlock(&list_mutex);
return bytes_read;
}

static ssize_t dev_write(struct file *filep, const char *buffer, size_t
len, loff_t *offset)
{
    struct data_node *new_node;

    new_node = kmalloc(sizeof(struct data_node), GFP_KERNEL);
    if (!new_node)
        return -ENOMEM;

    new_node->message = kmalloc(len + 1, GFP_KERNEL);
    if (!new_node->message)
    {
        kfree(new_node);
        return -ENOMEM;
    }

    if (copy_from_user(new_node->message, buffer, len))
    {
        kfree(new_node->message);
        kfree(new_node);
        return -EFAULT;
    }

    new_node->message[len] = '\0';
    new_node->size = len;

    if (mutex_lock_interruptible(&list_mutex))
    {
        kfree(new_node->message);
    }
}

```

```
        kfree(new_node);
        return -ERESTARTSYS;
    }

    list_add_tail(&new_node->list, &msg_list);

    mutex_unlock(&list_mutex);

    printk(KERN_INFO "ListDriver: Dado adicionado (%zu bytes)\n", len);
    return len;
}

module_init(list_driver_init);
module_exit(list_driver_exit);
```